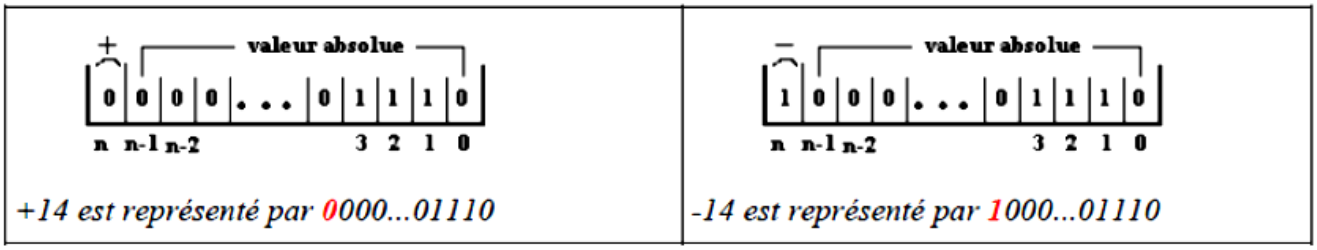
Le codage des entiers relatifs

I) REPRESENTATION :

Pour représenter les entiers relatifs en notation binaire, on doit étendre la représentation des positifs aux nombres négatifs.

Solution avec les premières machines :
Une solution est de réserver le bit de poids fort pour le signe de l'entier à représenter (1 pour – et 0 pour +) et d'utiliser les autres pour représenter sa valeur absolue :

Exemple du codage en binaire signé des nombres **+14** et **-14** :



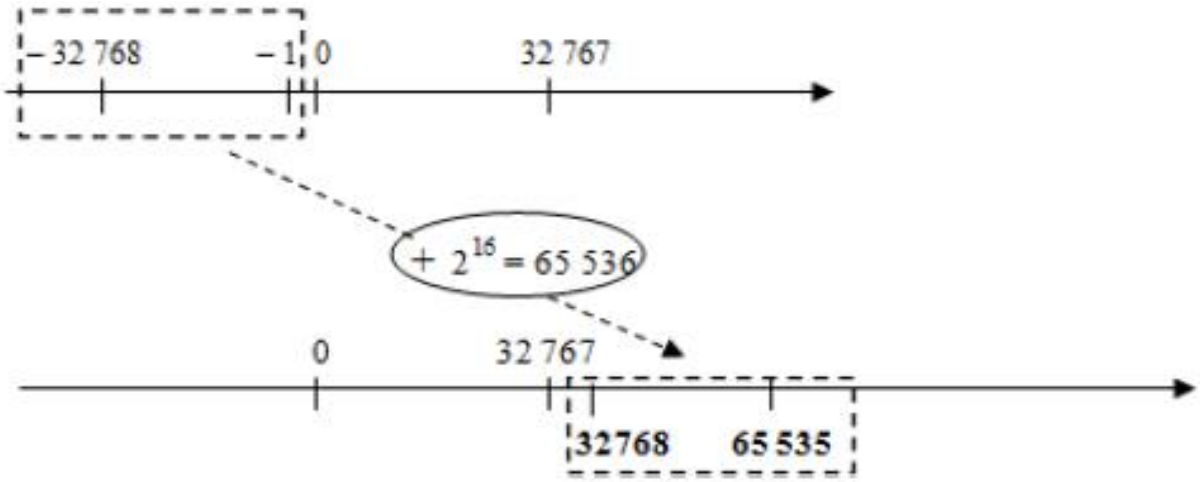
a) Ainsi, avec des mots de 16 bits, en utilisant 1 bit pour le signe et 15 pour la valeur absolue, quels sont les entiers que l'on peut représenter ?
b) Même question en 32 bits.

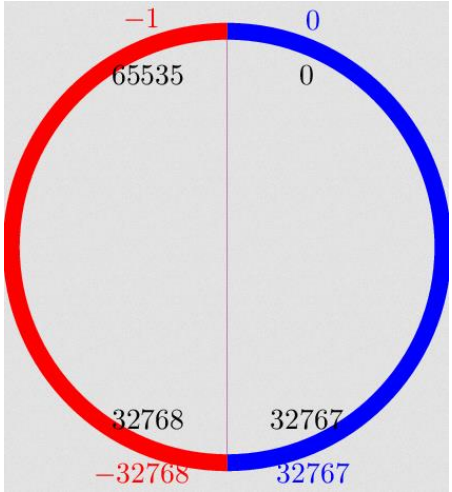
Ce codage n'est pas utilisé en pratique. En effet le traitement spécifique du signe coûte cher en circuits électroniques et en temps de calcul. C'est une version améliorée qui est utilisée dans la plupart des calculateurs : elle se nomme le complément à deux.

II) LE COMPLEMENT A 2 (C2) :

1°) Première approche :
Si on utilise des mots de 16 bits, les nombres $(1000\ 0000\ 0000\ 0000)_2$ et $(0000\ 0000\ 0000\ 0000)_2$ représentent 2 nombres distincts donc on peut représenter les entiers relatifs compris entre $-2^{15} = -32\ 768$ et $2^{15} - 1 = 32\ 767$. L'idée est de transporter les entiers strictement négatifs dans le monde des entiers naturels et de laisser intact les entiers positifs (voir ci-dessous) :

Avec cette méthode, tous les entiers relatifs sont donc représentés par un entier naturel.





On peut donc résumer la situation par le schéma ci-contre.

Si on utilise des mots de 16 bits, on peut représenter les entiers relatifs compris entre -32 768 et 32767. Donc sur 16 bits, on représente un entier relatif z positif ou nul comme l'entier naturel x et un entier relatif r strictement négatif comme l'entier naturel $x + 2^{16} = x + 65536$, qui est compris entre 32 768 et 65 535. Ainsi, les entiers naturels de 0 à 32 767 correspondent aux entiers relatifs positifs ou nuls, à droite sur la figure et les entiers naturels de 32 768 à 65 535 représentent les entiers relatifs strictement négatifs à gauche sur cette même figure.

Finalement on peut résumer la situation ainsi : dans certains intervalles fixés par les limites imposées par le nombre de bits disponibles noté Nb, les entiers relatifs sont représentés par leur valeur modulo 2^{Nb}.

Lorsque l'on s'intéresse au C2, il faut d'abord s'entendre sur le nombre de bits sur lequel on travaille. Certains processeurs (plutôt anciens) travaillent sur des représentations sur 8 bits. Dans ce cas les entiers relatifs vont de -2^7 à $2^7 - 1$.

a) Comment est alors représenté l'entier négatif –1 ?
b) Quels entiers relatifs peut-on représenter avec des mots de 8 bits ? Combien sont-ils ?
c) Mêmes questions pour 32 et 64 bits.

La représentation binaire sur n bits d'un entier relatif donné en décimal est :

- Si l'entier relatif x est positif ou nul, on le représente comme l'entier naturel x.
- S'il est strictement négatif, on le représente comme l'entier naturel $x + 2^n$.

Sur 8 bits :
L'entier relatif 0 est représenté comme l'entier naturel 0 soit $(0000\ 0000)_2$.
L'entier relatif –128 est représenté comme l'entier naturel $-128 + 2^8 = -128 + 256 = 128$ soit $(1000\ 0000)_2$.

d) Trouver la représentation binaire sur 8 bits des entiers relatifs 127 et –127.

La représentation décimale d'un entier relatif donné en binaire sur n bits :
Si cet entier relatif est donné par le mot m, on commence par calculer l'entier naturel p représenté par ce mot.

- Si p est strictement inférieur à 2^{n-1}, c'est l'entier relatif représenté,
- S'il est supérieur ou égal à 2^{n-1}, l'entier relatif représenté est $p - 2^n$.

Sur 8 bits :
Le mot $(0000\ 0000)_2$ représente l'entier naturel 0 et donc l'entier relatif 0.
Le mot $(1000\ 0000)_2$ représente l'entier naturel $128 = 2^7$ et donc l'entier relatif $128 - 2^8 = 128 - 256 = -128$.

e) Trouver la représentation décimale des entiers relatifs dont la représentation binaire sur huit bits est $(0111\ 1111)_{C2}$ et $(1000\ 0001)_{C2}$.

2°) <u>Un peu de théorie</u> :

En informatique, le **complément à deux** est une opération sur les nombres binaires qui permet d'effectuer simplement des opérations arithmétiques sur les entiers relatifs.
Le complément à deux ne s'applique qu'à des nombres ayant tous la même longueur : codés sur N bits, les nombres binaires peuvent représenter les valeurs entières de -2^{N-1} à $2^{N-1} - 1$.

<u>Description</u> :
Le complément à deux opère toujours sur des nombres binaires ayant le même nombre de bits (par commodité, celui-ci est généralement un multiple de 4). Dans une telle écriture, le bit de poids fort (bit le plus à gauche) donne le signe du nombre représenté (positif ou négatif). C'est le bit de signe.
Une représentation naïve pourrait utiliser ce bit de poids fort comme marqueur du signe, les autres bits donnant une valeur absolue :
00000010 = +2 en décimal
10000010 = −2 en décimal
Cette représentation possède deux inconvénients. Le premier (mineur) est que le nombre zéro (0) possède deux représentations : 00000000 et 10000000 sont respectivement égaux à +0 et −0. L'autre inconvénient (majeur) est que cette

représentation impose de modifier l'algorithme d'addition ; si un des nombres est négatif, l'addition binaire usuelle donne un résultat incorrect. Ainsi :
00000011 + 10000100 = 10000111
Soit 3 + (−4) = (−7) au lieu de (−1).
C'est pour remédier à ces problèmes que l'on utilise la notation en **complément à deux**. Les nombres positifs sont représentés comme attendu, en revanche les nombres négatifs sont obtenus de la manière suivante :

- On inverse les bits de l'écriture binaire de sa valeur absolue (opération binaire NON), on fait ce qu'on appelle le complément à un ;
- On ajoute 1 au résultat (les dépassements sont ignorés).

Cette opération correspond au calcul de $2^n - |x|$, où n est la longueur de la représentation et $|x|$ la valeur absolue du nombre à coder. Ainsi, −1 s'écrit comme 256 − 1 = 255 = $(11111111)_2$, pour les nombres sur 8 bits. Ceci est à l'origine du nom de cette opération : « complément à 2 puissance *n* », quasi-systématiquement tronqué en « complément à 2 ».
Les deux inconvénients précédents disparaissent alors. En effet, l'opération précédente effectuée sur 00000000 permet d'obtenir 00000000 (−0 = +0 = 0) et l'addition usuelle des nombres binaires fonctionne.
La même opération effectuée sur un nombre négatif donne le nombre positif de départ : $2^n - (2^n - x) = x$.
Pour coder (−4) :

- on prend le nombre positif 4 : 00000100 ;
- on inverse les bits : 11111011 ;
- on ajoute 1 : 11111100.

Le bit de signe est automatiquement mis à 1 par l'opération d'inversion. On peut vérifier que cette fois l'opération 3 + (−4) se fait sans erreur :
00000011 + 11111100 = 11111111
Le complément à deux de 11111111 est 00000001 soit 1 en décimal, donc 11111111 = (−1) en décimal.
Le résultat de l'addition usuelle de nombres représentés en complément à deux est le codage en complément à deux du résultat de l'addition des nombres. Ainsi les calculs peuvent s'enchaîner naturellement.
Si l'on doit transformer un nombre en son complément à deux « de tête », un bon moyen est de garder tous les chiffres depuis la droite jusqu'au premier 1 (compris) puis d'inverser tous les suivants.

- Prenons par exemple le nombre 20 : 00010100.
- On garde la partie à droite telle quelle : (00010**100**).
- On inverse la partie de gauche après le premier un : **11101**100.
- Et voici −20 : 11101100.

<u>Explications détaillées</u> :
D'un point de vue plus technique, cette écriture est simplement la troncature de l'écriture infinie à gauche. Pour la base 10, on sait qu'il est sans effet de compléter un nombre par des zéros à sa gauche, i.e. 123 peut s'écrire 0123, 00123, 000123, etc, avec une infinité de 0 à sa gauche.
De même, si on considère une infinité de 9 à gauche on obtient une représentation des nombres négatifs. Par exemple :

```
  …9999 (infinité de 9 à gauche)
+ …0001 (infinité de 0 à gauche)
   -------
 …0000 (infinité de 0 à gauche)
```

On peut alors interpréter …9999 comme étant −1, puisque −1(i.e. …9999) + 1 = 0.
Cette notation est le complément à 10. Pour obtenir la représentation d'un nombre négatif il faut complémenter à 9 chaque chiffre puis ajouter 1 au résultat. Ainsi pour obtenir la représentation de -123 on fait : …0123 transformé en …9876 puis en …9877.

Un exemple plus complet. Essayons de calculer dans une telle représentation 12 + (−7). 12 s'écrit …012, −7 s'écrit (…07 complémenté en …92 puis additionné de 1 donne …93) …93. Additionnons :

```
  …012
+  ….93
 --------
   ….05 Et chacun sait que 12 + (−7) = 12 − 7 = 5.
```

Une telle écriture mais de taille fixe fonctionne car le chiffre le plus à gauche (le signe 0 pour le + et 9 pour le −) représente alors simplement l'infinité des chiffres à gauche (l'opération consistant à allonger à volonté l'écriture du nombre à gauche s'appelle l'extension du signe et est bien connue des informaticiens).
Le complément à deux est alors la même technique employée avec la base 2.

On travaille sur k bits et on représente $\{-2^{k-1},...,2^{k-1}-1\}$.
Le complément à deux d'un entier n est obtenu par le calcul de la quantité $2^k - n$. En effet, "inversion" et " + 1" revient à faire $((2^k - 1) - n) + 1$.
Attention à l'ordre : " + 1" et ensuite "inversion" revient à faire $(2^k - 1) - (n + 1) = 2k - (n + 2)$
Le complément à deux du complément à deux d'un entier n est l'entier lui-même. En effet, $2k - (2k - n) = n$
Le complément à deux de zéro est zéro. En effet, $2k - 0 = 2k$, on néglige la retenue qui dépasse la taille fixée.
Le nom complet de cette opération est « complément à 2k » qui est en général tronqué en « complément à 2 ».

<u>Démonstration</u> : Soit N un entier positif codé sur n bits et $a_{n-1}\cdots a_1a_0$ sa décomposition binaire.
Notons $comp_2(N)$ son complément à 2.
On a $N = (\sum_{k=0}^{n-1} a_k.2^k)$ donc : $comp_2(N) = 2^n - (\sum_{k=0}^{n-1} a_k.2^k) = 1 + \sum_{k=0}^{n-1} 2^k - \sum_{k=0}^{n-1} a_k.2^k$ car $2^n = 1 + \sum_{k=0}^{n-1} 2^k$
(Par exemple, sur n = 8 bits, $\sum_{k=0}^{n-1} 2^k = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$, donc $1 + \sum_{k=0}^{n-1} 2^k = 256 = 2^8$).

Soit $comp_2(N) = 1 + \sum_{k=0}^{n-1}(1 - a_k).2^k$, en mettant 2^k en facteur.

Avec $(1 - a_k)$ qui représente donc le complément à 1 du bit a_k.

<u>On a donc la méthode pour la conversion en C2</u> :
<u>Pour un entier positif</u> : on prend son code binaire. <u>Ex</u> : $(120)_{10} = (01111000)_2 = (01111000)_{C2}$.
<u>Pour un entier négatif</u> : on remplace tous les 1 de sa représentation binaire par des 0 et vice-versa (c-à-d en remplaçant la valeur a de chaque bit par 1 − a) puis ajouter 1 (en ne gardant, si besoin) que les n premiers bits).
<u>Ex</u> : $(-58)_{10}$ on a $(58)_{10} = (00111010)_2$, donc $(-58)_{10} = (11000110)_{C2}$.

<u>Astuce</u>
Pour transformer de tête un nombre binaire en son complément à deux, on parcourt le nombre de droite à gauche en laissant inchangés les bits jusqu'au premier 1 (compris), puis on inverse tous les bits suivants. Prenons comme exemple le nombre −20. On prend sa valeur absolue (20) : $(00010100)_2$.
1. On garde la partie à droite telle quelle : 00010**100**
2. On inverse la partie de gauche après le premier un : **11101**100
3. Et voici −20 : $(11101100)_{C2}$.

Entrainez-vous à faire des conversions base 10 → C2, et le contraire. Dans ce domaine, la pratique est importante.
Pour vérifier vos résultats : https://fr.planetcalc.com/747/

<u>On peut en rire, ou en pleurer…</u>
Le 4 juin 1996, une fusée Ariane 5 a explosé 40 secondes après l'allumage. La fusée et son chargement avaient coûté 500 millions de dollars. La commission d'enquête a rendu son rapport au bout de deux semaines. Il s'agissait d'une erreur de programmation dans le système inertiel de référence. À un moment donné, un nombre codé en virgule flottante sur 64 bits (qui représentait la vitesse horizontale de la fusée par rapport à la plate-forme de tir) était converti en un entier sur 16 bits. Malheureusement, le nombre en question était plus grand que 32768 (le plus grand entier que l'on peut coder sur 16 bits) et la conversion a été incorrecte.

III) <u>ET PYTHON DANS TOUT ÇA ?</u> :

1°) <u>Des conversions avec Python</u> :
Python possède des modules et des fonctions qui permettent des changements de bases automatiques.
Observer le fonctionnement des commandes suivantes dans la console.

```
>>> a=42
>>> b=1200
>>> print(a)
>>> print("{}".format(a))
>>> print("{}".format(b))
```

On obtient le résultat :

```
42
42
1200
```

LA METHODE FORMAT.

La méthode format propose plus d'options pour formatter le texte et son usage est de plus en plus fréquent. La méthode interprète les accolades {} comme des codes qu'elle remplace avec les valeurs passées en argument. Le type n'importe plus. Quelques exemples :

```
>>> print('{0}, {1}, {2}'.format('a', 'b', 'c'))   # le format le plus simple
>>> print('{}, {}, {}'.format('a', 'b', 'c'))      # sans numéro
>>> print('{2}, {1}, {0}'.format('a', 'b', 'c'))   # ordre changé
>>> print('{0}{1}{0}'.format('abra', 'cad'))       # répétition
```

Résultat :

```
a, b, c
a, b, c
c, b, a
abracadabra
```

La méthode accepte aussi les paramètres nommés et des expressions.

```
>>> print('Coordinates: {latitude}, {longitude}'.format(latitude='37.24N', longitude='-115.81W'))
>>> coord = (3, 5)
>>> print('X: {0[0]};  Y: {0[1]}'.format(coord))
```

Résultat :

```
Coordinates: 37.24N, -115.81W
X: 3;  Y: 5
```

L'alignement est plus simple :

```
>>> print('A{:<30}B'.format('left aligned'))
>>> print('A{:>30}B'.format('right aligned'))
>>> print('A{:^30}B'.format('centered'))
>>>print('A{:*^30}B'.format('centered'))
```

Résultat :

```
Aleft aligned                  B
A                right alignedB
A           centered           B
A***********centered***********B
```

Format numérique (très utile pour les changements de bases) :

```
>>> print('{:.2%}'.format(19.0 / 22.0))
>>> print("int: {0:d};  hex: {0:x};  oct: {0:o};  bin: {0:b}".format(42))
>>> print("int: {0:d};  hex: {0:#x};  oct: {0:#o};  bin: {0:#b}".format(42))
>>> print('{:,}'.format(1234567890))
```

Résultat :

```
86.36%
int: 42;  hex: 2a;  oct: 52;  bin: 101010
int: 42;  hex: 0x2a;  oct: 0o52;  bin: 0b101010
1,234,567,890
```

Dates :

```
>>> import datetime
>>> d = datetime.datetime.now()
>>> print('{:%Y-%m-%d %H:%M:%S}'.format(d))
```

Résultat :

```
2019-07-30 06:59:25
```

Le site pyformat recense d'autres usages de la méthode format comme l'affichage de chaînes de caractères tronquées.

```
>>> print('{:10.5}'.format('formatages'))
```

Résultat :

```
   forma
```

Il est donc possible de changer le format d'affichage. Dans ce qui suit, on fait afficher $(42)_{10}$ en base 2, et $(1200)_{10}$ en base 2.

```
>>> a= 42
>>> b= 1200
>>> print("{:b}".format(a))
101010
>>> print("{:b}".format(b))
10010110000
```

Si la base deux joue un rôle prépondérant en informatique, l'autre base également utilisée de façon fréquente est la base 16, dite hexadécimale. Elle permet de décrire ou de manipuler commodément une longue suite de bits. Il serait en effet vite difficile de manipuler des grandes séquences de bits, pour par exemple explorer un fichier ou entrer un code WIFI. On multiplierait ainsi les risques d'erreurs. Le cerveau humain est plus accoutumé à un faible nombre de symboles puisés parmi un grand choix plutôt qu'à une immense suite de symboles même s'ils sont puisés dans un ensemble de faible étendue.
Pour un affichage en hexadécimal :

```
>>> print("{:x}".format(a))
>>> print("{:x}".format(b))
```

Résultat :

```
2a
4b0
```

On peut aussi faire l'inverse :

```
>>> a=int("101010",2)
>>> print(a)
```

Résultat :

```
42
```

Testez avec b et avec d'autres bases.

```
>>> a=int("101010",2)
>>> print(a)
>>> print("{}".format(a))
>>> b=int("4b0",16)
>>> print(b)
>>> print("{}".format(b))
```

Résultat :

```
42
42
1200
1200
```

Il est même possible de formater la sortie en imposant un format précis. Dans ce qui suit, on impose le format binaire en 8 bits, et on remplit les éventuels places vides à gauche par des 0. :

```
>>> print("{0:{fill}8b}".format(a, fill='0'))
```

Résultat :

```
00101010
```

Que se passe-t-il si on souhaite afficher un négatif ? Si on tape (pour faire afficher - 42 en binaire sur 8 bits) :

```
>>> print("{0:{fill}8b}".format(-a, fill='0'))
```

On obtient :

```
-0101010
```

C'est donc ici que l'on voit toute l'utilité du C2, on représente ce nombre négatif par un nombre positif en C2.

a) Utiliser les formats d'affichage en Python afin de représenter a = 42 et b = - 42 sur 8 bits en C2 (se rappeler qu'en C2, les entiers relatifs sont représentés par leur valeur modulo $2^{\text{nombre de bits}}$).
b) Effectuer l'opération c = b + 1 et afficher en C2**.**
c) Afficher l'opposé de c : d = − c et afficher en C2.
d) Afficher l'addition de e = d + b et afficher en C2. Comment interpréter ce résultat ?
e) Passage à 32 bits. Refaire les mêmes calculs et affichages mais cette fois-ci sur 32 bits. Affichez les limites de représentations sur 32bits. Note : en Python l'exposant est noté : ** par exemple x^2 est codé ainsi : x**2.

f) Passage à 64 bits. Refaire la même chose cette fois-ci en format double précision (64 bits).

g) Comment inverser la conversion ? Vous devez partir d'une chaine représentant un nombre codé en C2 sur par exemple 8 bits : "11010111" ou encore "00101010"
Comment savoir si c'est un positif ou un négatif ?
Compléter ce code afin d'obtenir cet affichage à partir des mêmes chaînes en entrée.

```
print(int("11010111",2))
print(int("00101010",2))
....
```

Affichage :
215
42
-41
42

2°) Algorithmes de conversions :

Au fait, c'est quoi un algorithme ?

Selon Larousse :
Ensemble de règles opératoires dont l'application permet de résoudre un problème énoncé au moyen d'un nombre fini d'opérations. Un algorithme peut être traduit, grâce à un langage de programmation, en un programme exécutable par un ordinateur.

2°-1) Représenter en base k un entier naturel donné en base 10 :

Pour écrire les entiers naturels en base k, on a besoin de k chiffres. Quand on a r objets, on les groupe par paquets de k, puis on groupe ces paquets en paquets de paquets, etc. Autrement dit, on fait une succession de divisions par k, jusqu'à obtenir un quotient égal à 0.

Plus formellement, si l'on note $a_i,a_{i-1},...,a_1,a_0$ l'écriture de n en base k, est obtenue pour l'algorithme suivant :

entrées: n,k
$i \leftarrow 0$
tant que $n \neq 0$
faire
    $a_i \leftarrow$ reste de la division euclidienne de n par k
    $n \leftarrow$ quotient de la division euclidienne de n par k
    $i \leftarrow i+1$

Résultat : a_i, a_{i-1},..., a_1, a_0

Donner le code Python (implémentation) pour transformer en base 2 un nombre donné en base 10.

2°-2) Représenter en base 10 un entier naturel donné en base k :

Pour trouver la représentation en base dix d'un entier naturel donné en base k, on utilise le fait qu'en base k, le chiffre le plus à droite représente les unités, le précédent les paquets de k, le précédent les paquets de $k \times k = k^2$, le précédent les paquets de $k \times k \times k = k^3$, etc. On retrouve les opérations réciproques de celles effectuées dans l'algorithme de conversion en base k.

Si $a_i, a_{i-1}, ..., a_1, a_0$ est l'écriture de n en base k, alors : $n = a_0 + a_1 k + a_2 k^2 + ... + a_i k^i$

Ce calcul suppose $(i+1)+i+\ldots+1=(i+1)(i+2)/2$ produits.

La fonction "int" ne permet pas de dépasser la base 36.

a) Ecrire une fonction Python permettant la conversion d'un nombre écrit en base 2, à sa valeur en base 10.

b) Écrire une fonction prenant en paramètres une chaîne de caractères s et une base k inférieure à 64 et retournant l'entier dont l'écriture en base k est s.

Coder cet algorithme de deux façons : dans un cas le parcours est effectué dans l'ordre de la chaîne et dans l'autre le parcours est effectué dans l'ordre inverse.

Aide : Pour avoir le rang du caractère c dans digits vous pouvez utiliser : digits.index(c)